

---

# Coroutine Executors and P2464R0

---

Document Number: P4062R0  
Date: 2026-03-14  
Audience: LEWG, SG1  
Reply-to: Vinnie Falco [vinnie.falco@gmail.com](mailto:vinnie.falco@gmail.com)

## Table of Contents

---

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. P2300R10

3. Two Signatures

4. The Error Channel

5. After Submission

6. Composition

7. Structured Concurrency

8. The Executor Is Infrastructure

9. Freedom and Guarantees

10. Non-Template Operation States

11. The Trade-Off

12. What the Trade-Off Buys Us

Acknowledgments

References

## Abstract

---

P2464R0<sup>[1]</sup> identified three properties of P0443 executors: no error channel, no lifecycle for submitted work, and no generic composition. All three were correct. This paper checks whether those properties hold for the coroutine-native executor described in P4003R0<sup>[2]</sup>. They do not. The coroutine executor concept did not exist in its current form until P4003R0<sup>[2]</sup> was published in 2026; this analysis became possible only then. The remainder documents what the coroutine constraint makes possible.

---

## Revision History

---

R0: March 2026 (post-Croydon mailing)

- Initial version.

## 1. Disclosure

---

The authors developed and maintain Corosio<sup>[5]</sup> and Cappy<sup>[4]</sup> and believe coroutine-native I/O is the correct foundation for networking in C++. The authors provide information, ask nothing, and serve at the pleasure of the chair.

---

## 2. P2300R10

---

P2464R0<sup>[1]</sup> critiqued P0443. P2300R10<sup>[6]</sup> addressed those deficiencies with the sender/receiver model. P4003R0<sup>[2]</sup> provides a second async model. The committee now has two implementations to evaluate against P2464R0<sup>[1]</sup>'s criteria. This paper evaluates P4003R0<sup>[2]</sup> and uses P2300R10<sup>[6]</sup> as a reference. Structural comparisons are observations, not arguments that `std::execution` should be modified or removed.

---

## 3. Two Signatures

---

P0443<sup>[3]</sup>:

```
void execute(F&& f);
```

Coroutine executor ([P4003R0](#)<sup>[2]</sup>):

```
std::coroutine_handle<> dispatch(  
    std::coroutine_handle<> h) const;  
  
void post(std::coroutine_handle<> h) const;
```

[P2464R0](#)<sup>[1]</sup> argued that these three deficiencies made P0443 inadequate as a foundation for async programming. [P2300R10](#)<sup>[6]</sup> addressed all three:

1. No error channel.
2. No lifecycle for submitted work.
3. No generic composition.

All three were correct for `execute(F&&)`. A coroutine executor is a type satisfying the `Executor` concept described in [P4003R0](#)<sup>[2]</sup>. This paper checks whether the three properties hold for `dispatch(coroutine_handle<>)` and `post(coroutine_handle<>)`.

---

## 4. The Error Channel

---

[P2464R0](#)<sup>[1]</sup>:

*“The only thing that knows whether the work succeeded is the same facility that accepted the work submission.”*

Apply that criterion. The coroutine executor accepts a `coroutine_handle<>`. It holds the handle. It knows.

`post(coroutine_handle<>)` has two call-site dispositions:

- Normal return. Ownership transferred. The executor holds the handle and is responsible for it.
- Exception. Submission rejected. The caller still owns the handle.

After ownership is transferred, the execution context has a third disposition: **destroy without resuming**. The awaiting coroutine is also destroyed, propagating cancellation upward through the coroutine tree. This is analogous to `set_stopped` in P2300R10<sup>[6]</sup>.

P0443's `execute(F&&)` had two dispositions: invoke or destroy without invoking. The coroutine executor separates call-site ownership from post-submission lifecycle, with explicit semantics for each. By P2464R0<sup>[1]</sup>'s own criterion, the coroutine executor satisfies the requirement. The facility that accepted the submission holds a typed resource with a defined lifecycle. It is not an opaque callable that vanishes on failure.

---

## 5. After Submission

---

P2464R0<sup>[1]</sup>:

*"Such existing practice has a huge problem that it just plain can't handle errors that occur after submission. That's an irrecoverable data loss."*

P0443's `execute(F&&)` takes an opaque callable. If the executor fails to invoke it, the callable is destroyed without being invoked. A well-written callable can detect this in its destructor. A `coroutine_handle<>` has the same property: `destroy()` without `resume()` is the signal.

A `coroutine_handle<>` is not an opaque callable. It has two operations: `resume()` and `destroy()`. The proposed wording specifies the ownership contract:

```
std::coroutine_handle<> dispatch(  
    std::coroutine_handle<> h) const;  
  
void post(std::coroutine_handle<> h) const;
```

**Preconditions:** `h` is a valid, suspended coroutine handle.

**Effects:** Arranges for `h` to be resumed on the associated execution context. For `dispatch`, the implementation may instead return `h`. For `post`, the coroutine is not resumed on the calling thread before `post` returns.

**Postconditions:** For `post`, ownership of `h` is transferred to the implementation. For `dispatch`, if `noop_coroutine()` is returned, ownership of `h` is transferred to the implementation; if `h` is returned, ownership of `h` is transferred to the caller via the return value and the implementation has no further

obligation regarding `h`. When ownership is transferred to the implementation, the implementation shall either resume `h` or destroy `h`. No other disposition is permitted.

**Returns** (*dispatch only*): `h` or `noop_coroutine()`.

**Throws:** `std::system_error` if the submission fails. If an exception is thrown, ownership of `h` is not transferred; the caller retains ownership.

**[Note:** The returned handle enables symmetric transfer. The caller's `await_suspend` may return it directly. - *end note*]

When the executor destroys a queued handle, the awaiting coroutine's `await_resume` is never called. If the parent coroutine is itself suspended in `when_all` or a similar combinator, the combinator detects the child's destruction and resumes the parent with an error. If no combinator is involved, the parent is destroyed in turn. In both cases, cancellation propagates upward through the coroutine tree. Parent outlives children. No handle is leaked.

---

## 6. Composition

---

P2464R0<sup>[1]</sup>:

*"How many different implementations of this function do I need to support 200 different execution strategies and 10,500 different continuation-decorations? With Senders and Receivers, the answer is 'one, you just wrote it.'"*

`co_await` is not the composition layer. It is the awaitable layer. The composition happens through what you

`co_await`:

```
copy::task<response> handle_request(stream& s) {
    auto [ec, n] = co_await s.read_some(buf);
    if (ec == copy::cond::eof)
        co_return make_response(buf, n);
    // ...
}
```

How many implementations? One. Works with any executor (inherited through `io_env` (P4003R0<sup>[2]</sup>)). Works with any stream (concept-constrained). Works with any caller (it is a `task<>` - anyone can `co_await` it). Completion handlers do not exist in this model.

I/O does not need the full algebra of concurrent composition. The dominant composition pattern in I/O code is a scoped timeout: do this operation, but cancel it after a deadline. That is `cancel_after` (Capy<sup>[4]</sup>). General-purpose combinators like `when_all` and `when_any` are available (Section 7), but `cancel_after` is what you get when you ask “what does I/O composition actually need?” and give a direct answer.

The `ioAwaitable` system (P4003R0<sup>[2]</sup>) is scoped in the sense that while inside it, the properties are invariant: executor affinity, stop token propagation, frame allocator policy. `std::mutex` is scoped (you hold the lock or you do not). `std::jthread` is scoped (one thread model). The `ioAwaitable` system is scoped in the same sense. You can leave at any time - `co_await` a bridge to senders, spawn a thread, call a blocking API. You can come back at any time - `co_await` an awaitable that bridges from the other model. The system is a region with guarantees. The guarantees hold because the design space is scoped. Step outside, the guarantees do not apply. Step back in, they resume.

The preceding sections address P2464R0<sup>[1]</sup>'s three criteria. The following sections document additional properties of the coroutine constraint.

---

## 7. Structured Concurrency

---

```
copy::task<dashboard> load_dashboard(  
    std::uint64_t user_id)  
{  
    auto [name, orders, balance] =  
        co_await copy::when_all(  
            fetch_user_name(user_id),  
            fetch_order_count(user_id),  
            fetch_account_balance(user_id));  
    co_return dashboard{name, orders, balance};  
}
```

```

capy::task<> timeout_a_worker()
{
    auto result = co_await capy::when_any(
        background_worker("worker"),
        capy::delay(100ms));

    if (result.index() == 1)
        std::cout << "Timeout fired\n";
}

```

## 8. The Executor Is Infrastructure

P2464R0<sup>[1]</sup>:

*"C++ programmers will write **thousands** of these executors."*

The coroutine executor concept has two operations:

```

std::coroutine_handle<> dispatch(
    std::coroutine_handle<> h) const;

void post(std::coroutine_handle<> h) const;

```

Both take a `coroutine_handle<>`. Both resume it on a context.

Three tiers of users:

1. Most users write `task<>` coroutines and pick an executor at the launch site. They never call `dispatch` or `post`.
2. Integration authors call `dispatch` / `post` when bridging foreign async mechanisms (databases, message queues, GPU completions).
3. Executor authors implement the concept.

Two operations. Both take `coroutine_handle<>`.

## 9. Freedom and Guarantees

A system's guarantees are the intersection of all conforming uses. Narrow the uses, widen the guarantees.

| Model                       | Design Freedom                                                                | Guarantees                                                                                                                                                    |
|-----------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P0443                       | Any callable, any executor                                                    | Almost none (no error channel, no result, no lifecycle)                                                                                                       |
| P2300 <sup>[6]</sup>        | Any receiver, any scheduler, eager or lazy                                    | Channel routing, structured concurrency, completion signatures, compile-time type safety, generic algorithms, lazy evaluation, zero-overhead composition      |
| ioAwaitables <sup>[2]</sup> | <code>coroutine_handle&lt;&gt;</code><br>+ <code>io_env const*</code><br>only | Executor affinity, stop propagation, frame allocator, concrete op states, type-erased streams, user-definable task types, ABI stability, separate compilation |

Some properties appear in both models. The table lists the guarantees each model provides by default, without additional machinery. ioAwaitables can achieve generic algorithms through sender bridges shipped in [Capy](#)<sup>[4]</sup>.

[P2300R10](#)<sup>[6]</sup> may be able to recover ABI stability through type erasure of operation states; this is an area of active exploration. The table describes what each model gives you without reaching for the other.

`std::mutex` is narrow (one locking model). `std::jthread` is narrow (one thread model). `std::stop_token` is narrow (one cancellation model). The standard contains narrow facilities alongside broad ones. Both belong.

## 10. Non-Template Operation States

A server opens 10,000 sockets. Each socket performs reads. How many operation state types exist?

Coroutine model - one:

```
struct read_op {
    OVERLAPPED overlapped;
    std::coroutine_handle<> h;
    io_env const* env;
    std::error_code ec;
    std::size_t bytes_transferred;
};
```

Not a template. Known at library-build time.



Sender model - one per distinct receiver type:

```
auto op = connect(
    socket.async_read(buf), my_receiver);
// decltype(op) depends on decltype(my_receiver)
```

Sender operation states are typically stack-allocated, not heap-allocated. The cost is not runtime allocation. The structural consequence of receiver-parameterization is:

- The implementation cannot go in a `.cpp` file. Separate compilation is foreclosed.
- The type is not known until `connect` is called with a specific receiver.
- The operation state cannot be embedded in the socket.
- The operation state cannot be reused across calls.

HALO (heap allocation elision) addresses coroutine frame allocation. It does not address the inability to compile sender operation states in a separate translation unit.

---

## 11. The Trade-Off

---

The coroutine constraint closes the design space at the I/O layer. Bridges exist at the boundary.

Consume a sender from coroutine-native code:

```
int result = co_await capy::await_sender(
    ex::schedule(sched)
    | ex::then([] { return 42; }));
```

Produce a sender from coroutine-native code:

```
auto sndr = capy::as_sender(
    capy::delay(500ms));
```

Both bridges ship in `Capy`<sup>[4]</sup>. The Networking TS gave users a choice of completion model. The coroutine executor removes that choice at the I/O layer, trading generality for the properties in Section 12. The choice remains available at the boundary.

---

## 12. What the Trade-Off Buys Us

---

| Property             | Concrete op state         | Template op state       |
|----------------------|---------------------------|-------------------------|
| Separate compilation | Yes ( <code>.cpp</code> ) | No (header-only)        |
| Pre-allocation       | Yes (in socket)           | No (type unknown)       |
| ABI stability        | Yes (known layout)        | No (receiver-dependent) |
| Contiguous pooling   | Yes (arrays)              | No (heterogeneous)      |

Concrete operation states produce these properties by default. Sender implementations can recover individual properties through type erasure; the table describes the default consequence, not an impossibility. A companion paper explores the full implications.

The header:

```
// handle_request.hpp
task<response> handle_request(any_stream& stream);
```

The implementation:

```
// handle_request.cpp
task<response> handle_request(
    any_stream& stream)
{
    auto [ec, req] =
        co_await http::read_request(stream);
    if (ec)
        co_return response::bad_request();
    auto res = process(req);
    std::tie(ec) =
        co_await http::write_response(
            stream, res);
    co_return res;
}
```

Type-erased. ABI-stable. Compiled once. Works with any transport. The header does not know whether the stream is a TCP socket, a TLS session, or a test mock. The `.cpp` file does not know who called it. The coroutine frame carries the state. The constraint made this possible.

---

## Acknowledgments

---

The authors thank Peter Dimov for identifying that P0443's callable destruction is detectable, correcting an earlier version of Section 5; Dietmar Kühl for `beman::execution` and ongoing work on P3552R3; Ville Voutilainen for P2464R0<sup>[1]</sup>, which provided the evaluation framework for this paper; Steve Gerbino and Mungo Gill for `Capy`<sup>[4]</sup> and `Corosio`<sup>[5]</sup> implementation work; and Klemens Morgenstern for Boost.Cobalt and the cross-library bridge examples.

---

## References

---

1. P2464R0 - "Ruminations on networking and executors" (Ville Voutilainen, 2021). <https://wg21.link/p2464r0>
2. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). <https://wg21.link/p4003r0>
3. P0443R14 - "A Unified Executors Proposal for C++" (Jared Hoberock, et al., 2020). <https://wg21.link/p0443r14>
4. `Capy` - Coroutine I/O foundation library (Vinnie Falco). <https://github.com/cppalliance/capy>
5. `Corosio` - Coroutine networking library (Vinnie Falco). <https://github.com/cppalliance/corosio>
6. P2300R10 - "std::execution" (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). <https://wg21.link/p2300r10>